

RetailMart Daily Sales ETL Pipeline

A robust, modular, and human-engineered ETL (Extract, Transform, Load) pipeline designed for **RetailMart Pvt. Ltd.** to ingest, clean, and transform multi-source daily sales data into production-ready analytical tables.

Project Overview

RetailMart collects daily transaction records across multiple Indian retail hubs. This pipeline automates the ingestion of raw, inconsistent CSV files, applies strict data-cleansing rules, computes business performance metrics, and stores the structured output in an optimized SQLite relational database for seamless business intelligence reporting.

Data Architecture Flow

1. Extract

- Ingests transactional data from CSV files.
- Loads product master sheets and store registry information.
- Reads data directly from local disk storage.

2. Transform

- Removes duplicate records.
- Handles missing values using structural imputation techniques.
- Standardizes schemas and data formats.
- Computes retail performance metrics and business KPIs.

3. Load

- Stores the transformed data into an SQLite database.
 - Creates production-ready analytical tables.
 - Enables efficient querying for dashboards and reporting.
-

Tech Stack

Component	Technology
Programming Language	Python 3.8+
Data Processing	Pandas, NumPy
Database	SQLite3
ORM (Optional)	SQLAlchemy

Project Structure

```
RetailMart-ETL-Pipeline/  
|  
├─ generate_data.py      # Generates sample retail datasets  
├─ pipeline.py          # Main ETL pipeline script  
├─ data/                # Raw CSV input files  
├─ output/              # Processed files and database  
├─ retailmart.db         # SQLite database  
├─ requirements.txt      # Project dependencies  
└─ README.md            # Project documentation
```

Installation

Clone the Repository

```
git clone https://github.com/your-username/RetailMart-ETL-Pipeline.git  
cd RetailMart-ETL-Pipeline
```

Install Dependencies

```
pip install pandas numpy
```

Or install using:

```
pip install -r requirements.txt
```

Running the Project

Step 1: Generate Sample Data

```
python generate_data.py
```

Step 2: Execute the ETL Pipeline

```
python pipeline.py
```